

ENTERPRISE DATA MODELLING & DATA CURATION

AI-Integrated Training Program · 16 Weeks · 45 Hours

FINAL ASSESSMENT

Part A – Quiz (30 Marks) · Part B – Capstone Assignment (70 Marks)

Field	Detail
Assessment Code	EDMDC/FA/01
Total Marks	100 (Part A: 30 Part B: 70)
Pass Criteria	Minimum 50 marks overall AND minimum 40% in each Part
Part A Duration	45 minutes
Part B Duration	10 calendar days from release
Mode	
Weightage	Contributes to the Final Capstone (30%) and Quiz (15%) program components
Prerequisite	

Participant Name: _____ Date: _____

1. General Instructions

Read this section completely before beginning either part of the assessment. Failure to follow the submission standards described here will result in a deduction under the Documentation and Repository Hygiene criterion.

1.1 Structure of the Assessment

Part	Component	Format	Marks	Time
1	Capstone assignment – UrbanCart Retail	9 practical tasks with code, models, dashboard and deployment	70	10 days
	TOTAL		100	

Your repository must follow this structure exactly:

```

_<Name>_UrbanCart/
  README.md           project summary, how to run, assumptions
  AI_USAGE.md         AI disclosure log (Section 1.4)
  01_architecture/    drawio source + PNG export
  02_data_model/      conceptual, logical ER, physical star schema
  03_sql/             ddl/ dml/ queries/ procedures/
  04_python/          notebooks + validation framework + config
  05_databricks/      bronze/ silver/ gold notebooks + job screenshot
  06_ai/              classification script, prompts, evaluation report
  07_powerbi/         .pbix file + page screenshots
  08_documentation/   data dictionary, lineage, security note
  09_deployment/      databricks.yml or deploy script, job config, runbook
  10_presentation/    10-slide deck

```

Part 1 – Capstone Assignment (70 Marks)

Duration: 10 days

2. Business Scenario – UrbanCart Retail Pvt. Ltd.

UrbanCart Retail operates 42 physical stores across nine Indian cities alongside a web store and a mobile application. Growth has outpaced the company's reporting capability. Point-of-sale exports arrive as daily CSV drops, the merchandising team maintains the product catalogue in a shared spreadsheet, clickstream events land as nested JSON, returns are logged in a separate legacy system, and a small share of orders are transacted in USD and SGD. There is no single source of truth. Two teams routinely present different revenue numbers for the same month, and roughly 30% of the catalogue has no category assigned, which makes category-level analysis impossible.

You have been engaged as the Data Modeller. Your mandate is to design and deliver a governed analytics platform that produces one trusted set of numbers, refreshes daily, and measures its own data quality.

2.1 Business Questions the Solution Must Answer

1. Net revenue by month, city, store format and sales channel.
2. Top 20 products by contribution margin, and each category's share of revenue.
3. Return rate by product category, with the dominant return reason for each.
4. Revenue split between new and repeat customers, month on month.
5. Average order value and average basket size, trended over the period.
6. A data quality scorecard by source system, showing completeness, validity and uniqueness.

2.2 Source Data Pack

Download `urbancart_raw.zip` from the LMS Resources folder. The data is deliberately imperfect; identifying and handling the defects is part of the assessment.

Source File	Approx. Rows	Known Characteristics
customers.csv	25,000	Duplicate customers under different emails; three date formats; inconsistent city spellings; invalid email and phone values
orders.csv	120,060	Order header – channel, store_id, order_status, currency (INR / USD / SGD), order_ts
order_items.csv	380,000	Line items – quantity, unit_price, discount_amount; a small number of negative quantities
products.csv	12,540	SKU, name, brand, category (approx. 30% NULL), cost_price, mrp
stores.csv	42	Store master – city, format (Flagship / High Street / Express), open_date
returns.csv	9,800	Free-text return reason requiring standardisation
web_events.json	200,000	Nested clickstream JSON – session, event type, device, product viewed
Frankfurter FX API	–	Public endpoint, no API key. Used to convert USD and SGD orders to INR at the order date. Endpoints and caveats in the data pack README

3. Tasks and Mark Allocation

Task	Description	Marks
1	Business case and solution architecture	7
2	Data modelling – conceptual, logical and physical	13

Task	Description	Marks
3	SQL Server implementation and advanced SQL	11
4	Python data curation, validation and API integration	11
5	Databricks medallion pipeline and Delta Lake	8
6	AI-powered data quality and automation	5
7	Power BI dashboard and insight storytelling	5
8	Governance, documentation and repository hygiene	3
9	Databricks deployment and operationalisation	7
	TOTAL – PART B	70

Task 1 – Business Case and Solution Architecture (7 marks)

- Write a one-page problem statement covering the business problem, project scope, explicit out-of-scope items, and at least five stated assumptions.
- Produce a solution architecture diagram in draw.io showing every source, the ingestion mechanism, the Bronze / Silver / Gold layers, the serving layer in SQL Server, and the Power BI consumption layer. Label the tool used at each hop.
- Justify two design decisions in writing – for example, why Delta format rather than Parquet for the Silver layer, and why the Gold layer is served from SQL Server rather than queried directly from the lakehouse.
- Commit both the .drawio source and a PNG export to 01_architecture/.

Task 2 – Data Modelling: Conceptual, Logical and Physical (13 marks)

- Conceptual model: identify the core business entities and their relationships, with cardinality. No attributes required at this level.
- Logical model: a full ER diagram for the operational layer, normalised to 3NF. Mark all primary and foreign keys and show relationship cardinality.
- Physical dimensional model: at least two fact tables (*FACT_SALES_LINE* and *FACT_RETURNS*) and at least five dimensions (*DIM_DATE*, *DIM_CUSTOMER*, *DIM_PRODUCT*, *DIM_STORE*, *DIM_CHANNEL*).
- **State the grain of each fact table explicitly in one sentence.** A model without a declared grain cannot score above half marks on this task.
- Implement Slowly Changing Dimension Type 2 on *DIM_PRODUCT* to track category reassignment. Document your surrogate key strategy and the effective-date columns.
- Justify your choice of star versus snowflake for this workload in no more than 150 words.
- Include a short naming convention standard (tables, columns, keys, indexes) and apply it consistently across the whole submission.

Task 3 – SQL Server Implementation and Advanced SQL (11 marks)

- Write DDL scripts that create the database, schemas and all tables with PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints, plus a justified indexing strategy.
- Load the Gold layer output into SQL Server and evidence the row counts against the source.
- Write six analytical queries that answer the business questions in Section 2.1. Between them the six must demonstrate: multiple join types, a CTE, aggregation with HAVING, and at least two window functions – including a ranking of the top products within each category and a month-on-month growth calculation using LAG.
- Create one view that exposes a reporting-ready sales summary, and one parameterised stored procedure, for example `usp_GetStorePerformance(@StoreID, @FromDate, @ToDate)`.

- Write a stored procedure that performs the SCD Type 2 merge into DIM_PRODUCT, handling inserts, updates and unchanged rows correctly.
- Every script must be idempotent – it should run twice without error and without duplicating data.

Task 4 – Python Data Curation, Validation and API Integration (11 marks)

- Profile every source file: row counts, null percentage per column, data types, and cardinality. Present the profile as a summary table in the notebook.
- Clean and standardise the data: resolve duplicate customers, unify the three date formats, standardise city names, validate emails and phone numbers with regular expressions, and detect price outliers with a stated statistical method.
- Flatten web_events.json into a tabular DataFrame, preserving the session and event hierarchy.
- **Build a config-driven validation framework.** Rules – not-null, range, regex, referential integrity and at least one cross-field rule – must be defined in an external configuration file, not hard-coded. Failed records are written to a quarantine file with a reason code, and every run writes a log line recording rules executed, records passed and records rejected.
- Call the Frankfurter FX API to convert USD and SGD orders to INR. Convert at the order date rather than today's rate; handle the fact that no rate is published on weekends and holidays; handle non-200 responses and retry transient failures. Commit a Postman collection evidencing the call.
- Produce three EDA charts with matplotlib or seaborn, each accompanied by a one-line observation.

Task 5 – Databricks Medallion Pipeline and Delta Lake (8 marks)

- Bronze: ingest all sources as received with no business transformation, adding audit columns for ingestion timestamp and source file name.
- Silver: apply the cleansing and conforming logic, deduplicate, enforce data types, and apply the validation rules from Task 4.
- Gold: materialise the dimensional model defined in Task 2 as Delta tables.
- Use MERGE INTO for at least one upsert. Demonstrate time travel by querying an earlier version and showing the difference. Add a written note on schema evolution and on when you would run OPTIMIZE and VACUUM.
- Evidence the row count at each layer – Bronze, Silver and Gold – and reconcile it back to the source files, accounting for every row that was dropped or quarantined.

Task 6 – AI-Powered Data Quality and Automation (5 marks)

- Use an LLM API to assign a category to the products where category is NULL, choosing only from the existing taxonomy.
- Design the prompt for reliability: constrain the output to a JSON schema, supply the allowed category list, include few-shot examples, and state and justify your temperature setting.
- Return a confidence score per prediction. Manually validate a random sample of 100 predictions and report the observed precision.
- Standardise the free-text return reasons into a fixed set of at most eight reason codes.
- **Document token usage, approximate cost, and your failure-handling approach. Confirm explicitly that no personally identifiable data was sent to the API – sending customer PII to an external model scores zero for this task.**

Task 7 – Power BI Dashboard and Insight Storytelling (5 marks)

- Connect to the SQL Server Gold layer and build the Power BI model with correct relationships, cardinality and filter direction, including a dedicated date table.

- Build three pages: Executive Summary, Product and Category Performance, and a Data Quality Scorecard.
- Write at least five DAX measures – Net Revenue, Average Order Value, Return Rate %, Repeat Customer %, and Month-on-Month Growth %.
- Include slicers, one drill-through page and at least one custom tooltip.
- Write five business insights derived from the dashboard, each paired with a specific recommended action.

Task 8 – Governance, Documentation and Repository Hygiene (3 marks)

- Produce a data dictionary covering every Gold table and every column: name, data type, description, source system, transformation applied and a PII flag.
- Draw a lineage diagram tracing at least one business metric from source column to dashboard visual.
- Write a security note covering your masking strategy for PII columns and a role-based access design for at least three roles.
- Deliver a complete README, a populated AI_USAGE.md, the prescribed folder structure, and a clean commit history as described in Section 1.3.

Task 9 – Databricks Deployment and Operationalisation (7 marks)

Tasks 1 to 8 produce a solution that works. This task asks whether it would survive being handed to somebody else. Everything below goes in 09_deployment/.

- **Environment separation.** Deploy the same code against two environments – for example urbancart_dev and urbancart_prod catalogs or schemas. Notebook widgets or job parameters must supply the environment, catalog, source path and target connection. Any hardcoded path, table name or connection string in a notebook forfeits the marks for this criterion.
- **Version control.** Link the workspace to your GitHub repository through Databricks Repos and commit notebooks as source files, not as an exported archive. State your branch strategy in one line.
- **Deployment artefact.** Commit something that recreates the job from configuration on an empty workspace – a Databricks Asset Bundle (databricks.yml) is preferred, but a CLI or Jobs API deployment script is accepted. A job created only by clicking through the UI does not qualify: the marker must be able to redeploy it from your repository.
- **Workflow orchestration.** Build a multi-task Databricks Job with explicit dependencies – ingest, Bronze, Silver, Gold, then load to SQL Server. Configure a job cluster rather than an all-purpose cluster, with a justified node type and autoscaling range, plus retries, a timeout and a schedule. Commit a screenshot of a successful multi-task run showing the duration.
- **Secrets.** Hold the SQL Server credentials and the LLM API key in a Databricks secret scope and read them with dbutils.secrets. Document the scope creation command. A credential visible in any committed notebook, config file or screenshot scores zero for this criterion and attracts the repository security deduction.
- **Reliability.** The job must be safely re-runnable – a second run on the same day must not duplicate data. Write a run audit table capturing run id, layer, rows read, rows written, rows quarantined and status for every execution.
- **Monitoring and rollback.** Configure a failure notification on the job. Document a rollback procedure using Delta time travel or RESTORE, and demonstrate it once by restoring a table to a prior version.
- **Runbook.** Write a one-page deployment runbook: prerequisites, deploy steps, how to verify a successful run, how to roll back, and the three checks somebody on call should perform if the 06:00 run fails.

4. Evaluation Rubric – Part 1

Each task is marked against the criteria below. Marks are awarded for a working, explainable solution – not for volume of code.

Task	Criterion	Marks
1	Problem statement, scope and assumptions	3
1	Architecture diagram completeness and correctness	2
1	Quality of design justification	2
2	Conceptual and logical ER model with correct cardinality and 3NF	4
2	Physical dimensional model, with grain declared for each fact	5
2	SCD Type 2 design and surrogate key strategy	3
2	Naming convention defined and applied consistently	1
3	DDL with complete constraints and justified indexing	3
3	Six analytical queries meeting the stated coverage	4
3	View and parameterised stored procedure	2
3	SCD Type 2 merge procedure, idempotent	2
4	Profiling, cleaning and standardisation	3
4	Config-driven validation framework with quarantine and logging	4
4	JSON flattening and REST API integration with error handling	3
4	EDA charts with observations	1
5	Bronze and Silver layers with audit columns and conforming logic	3
5	Gold Delta tables matching the Task 2 model	3
5	MERGE upsert, time travel demonstration and layer reconciliation	2
6	Prompt design, schema-constrained output and taxonomy adherence	2
6	Confidence scoring and precision measured on a sample	2
6	Cost, failure handling and PII safeguard documented	1
7	Correct Power BI model, relationships and date table	2
7	Three pages with required DAX measures and interactivity	2
7	Five insights, each with a recommended action	1
8	Data dictionary and lineage	1
8	Security and access design	1
8	README, AI disclosure, structure and commit history	1
9	Environment separation and fully parameterised code	2
9	Repos integration and a redeployable deployment artefact	2
9	Multi-task job, cluster config, retries, schedule and alerting	2
9	Secret scope, run audit table, rollback and runbook	1
	TOTAL	70

6. Assessment Checklist

Confirm each item before you submit.

- Repository follows the prescribed folder structure and is accessible to the instructor.
- All nine tasks attempted, with deliverables in the correct folder.
- Every SQL script and notebook runs end-to-end on a clean environment.
- Grain declared for each fact table.
- Validation rules held in an external configuration file, not in code.
- Screenshots included for the multi-task Databricks job run, Power BI pages and the Postman call.
- No credential, connection string or API key appears anywhere in the repository – all secrets read from the Databricks secret scope.
- The Databricks job can be redeployed from the committed artefact on an empty workspace.
- Deployment runbook committed, including the rollback procedure.
- Data dictionary covers every Gold column, with PII flagged.
- README explains how to run the solution from scratch.
- AI_USAGE.md completed.
- Minimum ten commits across at least three days.
- ZIP uploaded to LMS with the correct file name.